CandySpeak Manual

Tony Rogvall

2026-07-27

Contents

Introduction

CandySpeak is a reactive programming language designed for embedded systems and microcontrollers. It combines simplicity with powerful abstractions for handling sensors, timers, and I/O.

The language is rule-based - each line describes WHAT should happen and WHEN, not HOW. The system evaluates all rules every cycle and updates outputs based on inputs.

Core Concepts

- **Variables** - store values between cycles
- **Digital I/O** - buttons, LEDs, relays
- **Analog I/O** - sensors, PWM
- **Timers** - time-based events
- **Modules** - reusable components
- **States** - organise rules into state machines
- **Buffers & frames** - shared storage, bit-fields, pack/unpack
- **CAN** - a frame is a buffer with an address; fields are bit views into it

Language Reference

Declarations

Declarations start with # and define program resources.

Variables

```
#variable <name>[:<bits>] [<type>] [= <value>]
```

Examples:

```
#variable Counter:8 integer = 0
#variable Temperature float = 20.0
#variable Flag = 0
```

The width is in **bits, 1..32** (32 when omitted); anything else is refused, as is a literal that does not fit the type — a wrong width is far more expensive to find later than at the declaration.

> **A width without a type is SIGNED.** `#variable c:4` is a 4-bit *signed* value, so it counts `-8..7` and `c = 15` reads back as `-1`. That follows the rule that a typeless variable is `integer`, and it applies to `#field` and `bind` in exactly the same way. When you want the plain `0..2^N-1` range — flags, bit quantities, raw signals — say so:
>
> ```
> #variable c:4 unsigned = 15      // 15, not -1
> ```

Constants

```
#constant <name> = <value>
```

A constant is a named read-only value fixed at declaration. Unlike a variable it cannot be assigned by a rule; use it for thresholds, scale factors and pin-count limits that should read as names instead of magic numbers.

```
#constant Setpoint = 200
#constant Scale    = 4
```

Digital I/O

```
#digital <name> [in|out|inout] [pullup|pulldown] [<port>:]<pin>
```

Port is optional (for microcontrollers with multiple I/O ports).

Examples:

```
#digital Button in pullup 2
#digital Led out 13
#digital Relay out B:7
#digital Status inout C:4
```

Analog I/O

```
#analog <name>[:<resolution>] [in|out] [pwm] [<port>:]<pin>
```

Resolution is number of bits (default 10).

Examples:

```
#analog Sensor in A0
#analog Dimmer:8 out pwm 9
#analog HighRes:12 in A:1
```

Timers

```
#timer <name> <period_ms> [= 1]
```

`= 1` starts the timer automatically at program start.

Examples:

```
#timer Blink 500 = 1
#timer Debounce 50
#timer Timeout 5000
```

Buffers

```
#buffer <name>:<size> [<type>] [in|out] [can <frame-id>]
```

A buffer is a block of storage that variables can map into. A regular variable owns its own value; a buffer is *shared* — several variables can bind to different bit-fields of the same buffer, and the buffer can also be read and written directly by byte. Buffers are the building block for frames (CAN), bit packing, and overlapping data.

```
#buffer Frame:8            // 8 bytes of storage (64 bits)
#buffer Word:2             // 2 bytes
```

> **The size is always in bytes.** A `#buffer` is a byte container, whatever its transport — plain or CAN alike (a frame's size *is* its DLC, also bytes). Need a sub-byte masked value? That is a `#variable` (it carries its own bit width). Need a bit-view into a buffer? That is a `#field`, or a `bind` range.

A buffer may be declared inside a module, in which case every instance gets its own storage — see *What a Module May Contain*.

A buffer can be used directly like a variable. The whole buffer is its value (up to 4 bytes — larger buffers are accessed by byte or through bound fields):

```
#buffer Status:1
Status = 200
```

Byte access. Indexing a buffer selects whole bytes:

```
Frame[0] = 52          // first byte
Frame[1] = 18          // second byte
X = Frame[0..1]        // bytes 0..1 as one value (little-endian)
```

Binding variables (bit-fields). Map a variable onto a *bit* range of a buffer with `bind`. Writing or reading a bound variable reads/writes the underlying buffer bits — they alias the same storage. The buffer must be declared before it is bound.

```
#buffer Frame:3                          // 3 bytes = 24 bits of storage
#variable Speed:8     bind Frame[0..7]   // bits 0..7
#variable Rpm:10 big  bind Frame[8..17]  // bits 8..17, big-endian

Speed = 50             // writes into Frame's bits 0..7
```

> **Note:** when indexing a *buffer* directly the index is a **byte** (`Frame[1]`), while a `bind` range is in **bits** (`Frame[8..17]`). Direct indexing is for convenient whole-byte access; `bind` is for packing sub-byte bit-fields.

Either way the range must stay **inside the buffer** and cover at most **32 bits** — a bound field or a `Buf[a..b]` slice that reaches past the end, or asks for more than 32 bits, is refused at declaration time.

CAN frames

A frame is a buffer with an address. Declaring one adds a transport and a frame id to an ordinary buffer; everything else about buffers then applies unchanged — `bind`, `<<=`, `>>=`, byte indexing.

```
#buffer F201:8 in  can 0x201      // 8 BYTES (the DLC), received
#buffer Fbig:64 out can 0x300     // CAN FD, transmitted
```

The direction is the bus direction: `in` frames are received and unpacked, `out` frames are assembled and transmitted. Ids above `0x7FF` are sent as extended (29-bit) frames automatically.

Fields Signals inside a frame are bit views into it. There are three ways to reach them, and they are interchangeable — pick whichever reads best:

```
#buffer F201:8 in can 0x201

// 1. #field -- a named field, declared against the frame
#field Speed:16 unsigned F201[0..15]
#field Temp:8   unsigned F201[16..23]

// 2. bind -- the same thing spelled as a variable
#variable Rpm:16 bind F201[24..39]

// 3. >>= -- no declaration at all, unpack on the fly
F201 >>= a:8 b:8 c:16 ? F201.rx
```

A `#field` field inherits its direction from the frame, so it rarely needs one of its own. The bit range is in **bits**, counted from bit 0 of byte 0, and the value is little-endian unless the field says `big`.

Because all fields of a frame are views into one buffer, writing one field and reading another shows the packing directly:

```
#buffer Out:8 out can 0x200
#field Lo:8  unsigned Out[0..7]
#field Hi:8  unsigned Out[8..15]
#field Both:16 unsigned Out[0..15]

Lo = 1
Hi = 2                  // Both now reads 0x0201
```

Transmitting An `out` frame is sent at the end of any cycle in which one of its fields changed. That is an **event PDO** and needs no extra syntax — assigning a field is the trigger:

```
Speed = v                // frame 0x200 goes out this cycle
```

For a **cyclic PDO** — send on a schedule whether or not anything changed — there is nothing to make the frame dirty, so ask for the send explicitly with the `.tx` part:

```
#timer Beat 100
Beat = 1
Beat = 1      ? timeout(Beat)
Out.tx = 1    ? timeout(Beat)     // send every 100 ms regardless
```

`.dlc` controls how many bytes go out. It starts at the declared frame size and is clamped to it:

```
Out.dlc = 3              // send 3 bytes instead of 8
```

Receiving A received frame is delivered into the buffer and its fields update. `.rx` is true for **exactly one cycle** — the one in which the received bytes have become readable:

```
println("speed=", Speed) ? F201.rx
```

On the receive side `.dlc` reads back how many bytes the sender actually sent, and `.id` gives the frame id. Both work through the frame or through any field of it (`Speed.id` and `F201.id` are the same fact).

To react only when a *signal* changes rather than on every frame, guard on `changed()` instead — a frame that repeats its contents then prints nothing:

```
println("speed changed") ? changed(Speed)
```

> **One-cycle rule.** `.rx`, `changed()` and `<-` all fire in the cycle where the new value is still in the shadow copy, and rules read the *committed* side. Guarding a print on them directly therefore shows the **previous** value. Route the trigger through a variable to line them up — see *Execution Model*:
>
> ```
> #variable fresh = 0
> fresh = F201.rx
> println("speed=", Speed) ? fresh      // now in phase
> ```
>
> A `binded` variable does not have this problem: it *aliases* the frame bits rather than copying them, so it is already correct in the `.rx` cycle. An unpack (`>>=`) writes copies, which land next cycle like any other write.

Frame parts

Part	Direction	Meaning
`.id`	read	the CAN frame id

Part	Direction	Meaning
`.rx`	read	a frame arrived and is readable this cycle (one cycle only)
`.tx`	read/write	write 1 to force a send; reads back what was requested
`.dlc`	read/write	bytes to send / bytes last received
`.dir`	read/write	bus direction (`in` = 1, `out` = 2)

`.dir` is a property of the buffer, so it also works on a plain `#buffer`.

Note that frame parts live on the buffer rather than in a value slot, so unlike `.period` they are **not** shadowed: `F.dlc = 3` followed by a read of `F.dlc` in the same cycle gives 3.

Connecting to a bus On Linux the frames go to SocketCAN, selected with `--can`:

```
sudo ip link add dev vcan0 type vcan && sudo ip link set up vcan0
./csp --can=vcan0 -c 0 examples/can_input.csp
# from another shell:
cansend vcan0 201#2A3412785634120000
candump vcan0
```

Without `--can` the declarations still parse and the program still runs; frames simply go nowhere. On Arduino the backend is enabled per board by defining `CSP_HAS_CAN` in its Makefile (it expects the arduino-CAN API and a transceiver).

See `examples/can_input.csp`, `examples/can_output.csp` and `examples/can_pack.csp` for worked programs.

> **Limits today.** A field may start at any bit of a 64-byte FD frame (`0..511`) and may be **1..32 bits** wide — the value it produces is a 32-bit container, so a wider signal has to be split into two fields. A declaration outside those bounds, or one that reaches past the end of its frame, is refused rather than wrapped. RTR frames are not supported.

Packing and Unpacking Frames

`bind` gives a *named, permanent* field. When you just want to assemble or take apart a frame on the fly — without declaring a variable per field — use the pack (`<<=`) and unpack (`>>=`) operators. They are the compact, "really cool" way to build and decode frames.

Pack — `<buffer> <<= <field> <field> ...` writes several bit-fields into a buffer in one rule. Fields are **blank-separated** and fill **ascending** bit offsets; each is masked

to its width. A field is <expr>[:<bits>]; the width defaults to the declared width of the variable when omitted.

```
#buffer Frame:8
#variable A:3 = 5
#variable B:2 = 3
#variable C:3 = 6

Frame <<= A B C            // 5 | (3<<3) | (6<<5) = 221
```

Fields can be expressions and literals with an explicit width:

```
Frame <<= (X+4):3  X:2  6:3   // 5 | (1<<3) | (6<<5) = 205
```

Unpack — <buffer> >>= <var> <var> ... is the exact mirror: each variable receives the next bit-field from the buffer, low offset first.

```
#variable RA = 0
#variable RB = 0
#variable RC = 0

Frame >>= RA:3 RB:2 RC:3   // RA=5, RB=3, RC=6
```

Pack then unpack is a clean round-trip: <<= encodes, >>= decodes, and the widths line up field for field. Compared to bind, pack/unpack keep no state — they are a one-shot encode/decode you place in a rule, ideal for transient frame assembly just before sending or right after receiving.

Modules

Modules group related functionality for reuse. A module is a template that can be instantiated multiple times with different configurations.

```
#module <name>
  <declarations>
  <rules>
#end
```

Module Instantiation

```
#<ModuleName> <instance> [<init>]*
```

Where <init> can be:

Form	Meaning
field = value	Set the field's **value** (static, runs once)
field.part = value	Set a field **attribute** once — D.pin, D.port, T.period
field <- expr	Reactive connection (updates when expr changes)

Note the first two are different things: `D = 2` writes a value into `D`, while `D.pin = 2` places the pin. Configuring hardware is always the `.part` form.

Init expressions can be mixed freely. Fields marked `in` must be initialized.

Object Initialisation Semantics The two forms behave differently, by design:

- **Static (`=`, `.part =`) runs *once*.** These initialisers execute in the instance's implicit **INIT** state and then the object transitions to **NORMAL** (see *States*). Writing them every cycle would be wasteful and would keep configuration outputs permanently "dirty", so they are one-shot.
- **Reactive (`<-`) is a standing connection.** It re-evaluates whenever an input on its right-hand side changes. It is also **seeded once at start-up**: on the first cycle every `<-` fires once so the field gets an initial value — even when the right-hand side is a constant or a global that never changes afterwards. So `x <- G + 1` gives `x` the value `G + 1` immediately and then tracks later changes to `G`.

Fields versus globals. A `<-` initialiser may read a **global** freely:

```
#M m1 X <- G + 1          // G is a global -> fine
```

To express a relationship *between fields of an object* (or between two instances), write a rule rather than an initialiser — either inside the module, or globally using dot notation on the instances:

```
Total = m1.Out + m2.Out    // relate fields across instances with a rule
```

What a Module May Contain A module is a template. Inside `#module ... #end` you can declare the same resources as at top level — `#variable`, `#digital`, `#analog`, `#timer`, `#buffer`, `#field`, `#constant` — plus module-local `#states`, and the rules (including `#in <state>` blocks) that act on them. Each instance gets its own copy of every declared member and its own state, including its own buffer storage:

```
#module Frame
  #buffer B:16                        // one buffer PER INSTANCE
  #variable lo:8 bind B[0..7]
  #variable hi:8 bind B[8..15]
#end

#Frame f1 lo=1 hi=2                   // f1.B is 0x0201
#Frame f2 lo=9 hi=8                   // f2.B is 0x0809, independent
```

Modules can read globals. A module body sees everything declared above it — constants, variables, timers, buffers. A member with the same name **shadows** the global, so adding a global later cannot break a module that happens to use that name.

```
#constant GREEN = 0x07E0
#buffer Live:16                       // shared by every instance

#module Pixel
  #analog P out 9:0
  P = GREEN ? ...                     // global constant
```

```
  P = Live  ? ...                    // global buffer, one for all instances
#end
```

Binding a member to a *global* buffer therefore shares it across instances, while binding to a *member* buffer gives each instance its own — which of the two you get follows from where the buffer is declared.

Example: Full Adder

```
#module Add
#variable A:1 in
#variable B:1 in
#variable Cin:1 in
#variable S:1 out
#variable Cout:1 out

S = A ^ B ^ Cin
Cout = (A & B) | (Cin & (A ^ B))
#end

#Add a0 A=1 B=1 Cin=0
#Add a1 A=0 B=0 Cin <- a0.Cout
#Add a2 A=0 B=1 Cin <- a1.Cout
```

Here `a0` has static inputs, while `a1` and `a2` chain their carry input reactively from the previous adder's carry output.

Pin Assignment for I/O When a module contains `#digital` or `#analog` declarations, the pin can be left at a placeholder in the module and assigned per instance with the `.pin` part:

```
#module Button
#digital Pin in pullup 0     // placeholder pin
#variable Out = 0
#timer T 50

T = 1 ? Pin != Out
Out = Pin ? timeout(T)
#end

#Button btn1 Pin.pin=2       // assign the PIN at instantiation
#Button btn2 Pin.pin=3
#Button btn3 Pin.pin=4
```

> **`Pin = 2` is not the same thing.** A bare `field = value` sets the field's **value**, exactly as it does for a variable — for a 1-bit digital, `Pin = 2` writes the value 1 and leaves the pin at 0. Use `Pin.pin = 2` to place it, and `Pin.port = 1` for the port. The same applies to `#analog`.

This makes modules reusable across different hardware configurations. It also overrides a default given in the module:

```
#module Led
#digital Out out 13          // default pin 13
...
#end

#Led led1                    // uses default pin 13
#Led led2 Out.pin=12         // override to pin 12
```

Accessing Module Fields Use dot notation to access a module instance's fields:

```
MainLed = btn1.Out       // read debounced output
```

Rules

Rules have the form:

```
<actions> [? <condition>]
```

Actions are executed when the condition is true. The `? <condition>` part is optional — a rule with no condition runs every cycle (it is always true).

Assignment

Regular assignment - evaluated every cycle:

```
Led = 1 ? Button == 0
Counter = Counter + 1 ? Timer
```

Reactive Assignment

With `<-` the rule only runs when a variable on the right-hand side changes:

```
Output <- Input * 2
```

The rule above runs only when `Input` changes, not every cycle.

Start-up seed. A `<-` rule is also evaluated **once on the first cycle**, so the target always gets an initial value — even if the right-hand side is a constant or an input that never changes afterwards. This means `Output <- Input * 2` is correct from the very first cycle, not just after the next change to `Input`.

With condition - the rule runs when RHS variable changes AND condition is true:

```
Filtered <- RawValue ? RawValue > Threshold
```

Multiple Actions

Separate with comma:

```
Led = 1, Counter = Counter + 1 ? Button == 0
```

Conditions

Conditions can be:

- Comparisons: ==, !=, <, >, <=, >=
- Logical: && (and), || (or), ! (not)
- Timer functions: `timeout(T)`, `elapsed(T)`, `progress(T)`
- Special functions: `changed(x)`, `rising(x)`, `falling(x)`

Examples

```
Led = ~Led ? timeout(Blink)
```

Toggle LED at each timeout.

```
Output = 1, T = 1 ? Input && !Output
```

Set Output and start timer T when Input goes high.

States

States turn a flat list of rules into a state machine. You enumerate the states, then group rules under the state they belong to. This keeps large programs readable and lets the runtime skip whole blocks of rules that are not active.

```
#states A B C
```

Three states always exist implicitly: **INIT** (entered at start-up, for one-time initialisation), **NORMAL** (the default running state) and **FAILSAFE** (see below). Your own states are added on top; you can list more at any time with another `#states` line.

Rules are attached to a state with an `#in <state> ... #end` block:

```
#in A
    X = 1 ? Y < Z              // stay in A while this holds
    State = B ? X > Z          // transition to B
#end

#in B
    Z = Z + 1, State = A   // do work, then go back to A
#end
```

A transition is just an assignment to the reserved field `State`. Entering INIT code, doing initial setup, and moving on is the common shape:

```
#in INIT
    Count = 0, State = NORMAL
#end
```

Mental model. To reason about behaviour, read `#in S` as adding `&& State == S` to every rule in the block. The two forms below behave the same:

```
#in A
    Y = 1, State = B ? X > Z
```

```
#end
```

```
Y = 1, State = B ? X > Z && State == A
```

But it is a mental model, not a source rewrite. The block compiles to a **gate** in front of the group: one state test that jumps over the whole block when the state does not match, so an inactive state costs a single check no matter how many rules it holds — that is the point of grouping them. A per-rule state test is kept alongside it for the reactive path, which reaches rules individually rather than by walking the block.

You may add as many rules to a state as you like, and split a state across several `#in S` blocks — they accumulate.

Several states at once. List more than one state to run a block in *any* of them — the states OR together. Shared infrastructure (a heartbeat, a panic button, a timer restart) that must run across several phases goes here instead of being copied into each block:

```
#in red redyellow green yellow
    Phase = 1 ? timeout(Phase)     // restart the timer in every driving phase
    State = FAULT ? Panic          // the panic button, checked in all of them
#end
```

Read `#in A B C` as `&& (State == A || State == B || State == C)` on every rule in the block.

Rules with no `#in` — NORMAL+. A top-level rule that is *not* inside any `#in` block runs by default in the two built-in operating states, **INIT** and **NORMAL** — never in a special state. So a program with no states at all just runs (it sits in INIT/NORMAL), and the moment a state machine steps into a *special* state — a user state, or the reserved **FAILSAFE** — the loose global rules **quiesce** instead of leaking into it. That is what keeps FAILSAFE an island: only `#in FAILSAFE` (and blocks that explicitly list it) run there. To run a global rule in a special state too, name that state with `#in`.

FAILSAFE is sticky. It is the designated safe state, and once `State` holds it **no rule can leave it** — only a reset does. A rule that assigns something else is simply ignored, so a flaky guard cannot bounce the device back out of a safe configuration. Together with the quiescing rule above, that is what keeps FAILSAFE an island: loose global rules stop running there, and only `#in FAILSAFE` (plus blocks that name it) has any say over the outputs.

> This is the shape FAILSAFE has today, and it is deliberately thin. The intended form is a `#module FAILSAFE`, compiled as its **own ROM image** with its own declarations, its own `#in INIT` and its own EEPROM bank — so a device can carry several banks, one of them the safe one, and switching is a rebase rather than a state transition. Write safe-state logic as a self-contained block now and it will move over cleanly.

States in modules. A module can declare its own local states. Each instance carries its own `State`, so instances step through the machine independently:

```
#module Blinker
  #digital Led out
```

```
  #timer   T 500
  #states  on off

  #in INIT
    T = 1, State = off
  #end
  #in off
    Led = 1, State = on ? timeout(T)
  #end
  #in on
    Led = 0, State = off ? timeout(T)
  #end
#end

#Blinker b1 Led.pin=12
#Blinker b2 Led.pin=13
```

Parts

Every resource carries not just a *value* but a set of **attributes** — the pin of a digital, the period of a timer, the direction of a port. These attributes are reachable with dot-notation as **parts**, and most can be both read and written by a rule.

`<resource>.<part>`

Part	Applies to	Meaning
`.val`	any	the plain value (the default when no part is given)
`.pin`	digital / analog	pin number
`.port`	digital / analog	port number
`.dir`	digital / analog / buffer	direction (in/out)
`.pwm`	analog	PWM flag
`.endian`	bound field / `#field`	byte order (`0` native, `1` little, `2` big)
`.pullup`	digital	pull-up enable
`.pulldown`	digital	pull-down enable
`.period`	timer	timer period in ms
`.fired`	timer	timeout occurred this cycle
`.id`	CAN frame / field	the frame id
`.rx`	CAN frame / field	a frame arrived and is readable this cycle
`.tx`	CAN frame / field	write 1 to force a send
`.dlc`	CAN frame / field	bytes to send / bytes last received

A CAN field answers for its frame, so `Speed.id` and `F201.id` are the same fact. Frame parts are stored on the buffer rather than in a value slot, so unlike the others they

are **not** shadowed — writing `.dlc` and reading it back in the same cycle gives the new value.

Examples — reading and writing attributes like any other value:

```
D.pin    = 17              // reassign a digital's pin at runtime
T.period = 500             // change a timer's period
Per      = T.period        // read it back
Ready    = 1 ? T.fired     // use a timer part in a condition
```

Setting a part in object-init is the idiomatic way to configure an instance:

```
#Blinker b1 Led.pin = 12   // configure the instance's pin once, in INIT
```

Execution Model

A single cycle works like this.

- **Cycle.** The runtime repeatedly reads inputs, evaluates rules, and writes outputs. One pass is a *cycle*; `cycle()` returns its number.
- **Atomic commit.** Within a cycle, reads see the values committed at the end of the *previous* cycle, and writes are collected in a shadow copy. At the end of the cycle all changes are committed at once. So the order in which rules are written does not change the result, and a rule never sees another rule's half-finished update. When two rules write the same field in one cycle, the last write wins at commit.
- **Change set.** The commit records which fields actually changed. That is what drives `changed(x)` and the `<-` reactive trigger, and what the reactive execution mode uses to decide which rules to re-run.
- **Sequential vs reactive.** Sequential mode evaluates every rule each cycle. Reactive mode evaluates only rules whose triggers fired. The dependency graph is built from what stands behind `?`, and from both sides of `X <- Expr ? Cond`. A rule with neither a condition nor `<-` — a bare `Y = X` — therefore has **no trigger** and runs only in the first (seeding) cycle. That is by design, not a limitation: reactive mode runs what you told it to watch.

The one-cycle rule. Reads see the previous commit; writes land at the next one. Everything that *triggers on change* — `changed(x)`, `<-`, a frame's `.rx` — fires in the cycle where the new value is still in the shadow copy. So in that cycle the value itself still reads as the **old** one:

```
X = 1                      // X changes 0 -> 1
println(X) ? changed(X)    // prints 0, not 1
```

This is the transaction model applied consistently, not a special case: an input sampled this cycle becomes readable the next one, and that holds for a changed variable too. Two consequences worth knowing:

- To act on a change *with the new value*, route the trigger through a variable. It is delayed by exactly as much as the value, which puts them back in phase:

  ```
  #variable fresh = 0
  fresh = changed(X)
  ```

```
println(X) ? fresh          // prints 1
```

- A source that changes **exactly once** never re-triggers, so `Y <- X` captures the pre-change value and keeps it. With a continuously changing source the same mechanism just shows up as a one-cycle lag, which is harmless.

Values that *alias* rather than copy are exempt: a `binded` variable is the same storage as its buffer, so it is correct immediately.

Built-in Functions

Function	Description
`timeout(T)`	True for one cycle when timer T expires
`elapsed(T)`	Milliseconds since T started
`progress(T)`	0-100, percentage of timer period elapsed
`changed(x)`	True if x changed this cycle
`rising(x)`	True on 0→1 transition in digital input
`falling(x)`	True on 1→0 transition in digital input
`cycle()`	Current cycle number
`tick()`	Current time in milliseconds
`abs(x)`	Absolute value (integer)
`fabs(x)`	Absolute value (float)
`sign(x)`	Sign of x: -1, 0 or 1
`min(a,b)`	Minimum value
`max(a,b)`	Maximum value
`clip(x,lo,hi)`	Clamp x to the range lo..hi
`print(x)`	Print value (up to 4 arguments)
`println(x)`	Print with newline (0 to 4 arguments)
`latch(b)`	Hold outputs (1) or release (0)

Operators

Priority	Operator	Description
Highest	`~ ! -`	Bitwise NOT, logical NOT, negation
	`* / %`	Multiplication, division, modulo
	`+ -`	Addition, subtraction
	`<< >>`	Bit shift
	`< <= > >=`	Comparison
	`== !=`	Equality
	`&`	Bitwise AND
	`^`	Bitwise XOR
	`|`	Bitwise OR
	`&&`	Logical AND
Lowest	`||`	Logical OR

Linux Tool

Installation

```
git clone <repo>
cd candyspeak
make
```

Usage

```
./csp [options] [file.csp]
```

Options

Option	Description
`-h`	Show help
`-i`	Interactive mode
`-n`	Parse only, don’t execute
`-C`	Show compiled C code
`-c N`	Max number of cycles
`-T MS`	Max runtime in milliseconds
`-r`	Reactive mode (off unless given)
`-Q`	Trace variable values
`-P`	Debug parser
`-R`	Debug result
`-S`	Debug scanner/tokenizer
`-d`	Debug (general)
`-L erlang`	Output trace in Erlang format
`-s <file>`	Write a per-cycle state trace (Erlang format) to a file
`-p <file>`	Write parser debug output to a file
`-O <file>`	Write an object dump to a file
`-e <file>`	EEPROM (persisted state) file to use (default `eeprom.db`)
`--no-eeprom`	Do not overlay the saved EEPROM patches at boot
`-I <file>`	Feed inputs from a file (real time, cycle-stamped rows)
`-F <file>`	Feed inputs with **simulated time** (see below)
`-b`	Start paused; inspect, then `/resume` (implies `-i`)
`--can=IFACE`	SocketCAN interface for `#field` frames (e.g. `vcan0`)
`-m N[k]`	Usable code-memory budget in bytes
`-M N[k]`	Total RAM the simulated board has
`-U N[k]`	RAM the system and linked libraries take
`-E N[k]`	Simulated EEPROM capacity (0 = unbounded)
`--board=NAME`	Simulate a measured board: `mega`, `mkrzero`

Simulating a Board

The host tool can pretend to have a microcontroller's memory, so `/memory` reports numbers that mean something before you flash anything:

```
./csp --board=mega -i program.csp     # measured RAM/EEPROM for an ATmega2560
./csp -M 8k -U 2700 -i program.csp    # or set the numbers by hand
```

`--board` fills in `-M`, `-U` and `-E` from figures measured on real firmware builds (regenerate them with `make boards`). `/memory` then shows how much of that board's RAM the program would actually claim.

The EEPROM Is Read at Start-up

Started **without a program file**, `./csp` behaves like a board coming out of reset: it loads the baked-in ROM program and then overlays whatever `/save` wrote to the EEPROM file (`eeprom.db` unless `-e` says otherwise), so the declarations, rules and disables you saved last time are back. Give it a program file and the overlay is skipped — naming a file means "run *this*". `--no-eeprom` skips it too, for a deliberately clean boot or a repeatable test.

Examples

Run a program:

```
./csp program.csp
```

Show compiled code:

```
./csp -C -n program.csp
```

Run max 100 cycles with tracing:

```
./csp -c 100 -Q program.csp
```

Interactive mode:

```
./csp -i
```

Simulated Time (`-F`)

With `-F` the program runs against a **virtual clock** instead of the wall clock, which makes timer and input timing fully deterministic and instant (no real waiting). Each input row is stamped with an absolute virtual time in milliseconds:

```
<time_ms>  <var>=<value>  [<var>=<value> ...]
```

A row is applied once the virtual clock reaches its time. Between rows the clock jumps straight to the next event (the next input row or the next timer timeout), so a 1-second timer fires after one step rather than a thousand cycles — while still advancing at least one tick per cycle so `cycle()` and time-reading logic always see time move forward.

Example — feed a sensor at two virtual times and let a timer expire:

```
# stimulus.dat
30   Sensor=10
70   Sensor=21
```

```
./csp -c 100 -s trace.txt -F stimulus.dat program.csp
```

`csp_time_ms()` returns the virtual clock in this mode, so `timeout(T)`, `elapsed(T)` and `progress(T)` all behave deterministically. This is the recommended way to write repeatable tests for timers and analog/digital input.

Generate Embedded Code

To generate C code for Arduino or other microcontrollers:

```
./csp -C -n program.csp > program_code.h
```

Then include this header in your Arduino project along with `csp.h` and the runtime files.

Advanced Concepts

Execution Modes

By default every rule is evaluated each cycle. Reactive mode is an optional optimisation for large programs where few things change per cycle.

Regardless of mode, changes within a cycle are applied atomically at its end (see *Execution Model*): rules never see each other's half-finished updates and evaluation order does not matter.

Reactive Mode (`-r`)

Default: OFF — pass `-r` to turn it on.

In reactive mode, only rules whose inputs have changed are evaluated. This is more efficient when:

- Few variables change each cycle
- Program has many rules
- System has limited CPU

It costs some memory for the dependency graph.

What gets a trigger. The graph is built from what stands behind `?`, and from both sides of `X <- Expr ? Cond`. A rule that has neither a condition nor `<-` is not watching anything, so in reactive mode it runs only in the first (seeding) cycle:

```
Y = X            // sequential: every cycle. reactive: first cycle only.
Y <- X           // reactive: whenever X changes
Y = X ? cond     // reactive: whenever cond's inputs change
```

Write the rules the reactive way and both modes reach the same committed state. Rule order is honoured in both: when two rules write the same field, the one written last wins — which is what makes patching a running system predictable.

```
./csp -r program.csp     # reactive mode
./csp program.csp        # evaluate all rules (default)
```

Api

The support configuration and default values for reactive mode are contained in csp_config.h

```
#include "csp_config.h"
```

The configuration parameters are listed below. Each must be defined to either 1 or 0.

```
REACTIVE_DEFAULT
SUPPORT_REACTIVE

USE_STATISTICS
USE_FIXPOINT
```

The programmer's API to change these at runtime — for example in an Arduino sketch during `setup()`:

```
extern int     csp_set_reactive(csp_rt_t*, int onoff);
extern int     csp_set_latch(csp_rt_t*, int onoff);
```

Interactive Mode

Start interactive mode with `-i`:

```
./csp -i
./csp -i program.csp    # load program first
```

Interactive Commands

Command	Description
`/help`	Show commands
`/list`	List declarations and rules, tagged `[ROM]`/`[RAM]`
`/state`	Show current values, grouped per object
`/memory`	Show RAM usage per category (how much space is left)
`/reset`	Reset to initial values
`/clear`	Drop RAM patches, keep the ROM program
`/pause`	Freeze execution; the prompt stays live
`/resume`	Continue (rebuilds first if the program was edited)
`/live`	Freeze the *rules* but keep I/O running
`/latch on`	Hold outputs (freeze current values)
`/latch off`	Release outputs (normal operation)
`/commit`	Commit pending values

Command	Description
`/save`	Save state to storage (EEPROM file)
`/load`	Load state from storage (EEPROM file)
`/quit` (or `/exit`)	Exit

`/state` opens with a status line showing where you are:

```
cycle 214   latch on   running
```

The last field is the run mode — `running`, `paused` or `live`.

Pause and Live

`/pause` stops the cycle entirely: no input, no rules, no output. The prompt keeps working, so you can inspect state and add declarations or rules; the rebuild is deferred until `/resume`.

`/live` freezes only the **rules**. Input is still sampled and output is still written, so the hardware stays connected while the program stands still — which is what you want when you are poking at pins by hand:

```
/live
> Led = 1          # actually lights the LED
> Btn              # reads the real pin
/resume
```

`/resume` leaves both modes.

Editing a Running Program

Declarations and rules can be added at any time — running, paused or live. A new rule is wired into the reactive graph at the next cycle boundary, so it starts firing without a restart. This is the intended way to patch a live system: since the last rule to write a field wins, adding a rule at the prompt overrides an earlier one.

If a line fails to parse, nothing is kept. A failure inside an unfinished `#module` rewinds the **whole module** and reports `Module aborted`, so the lines you type next are not silently swallowed by a module that can never be closed.

Disabling and Enabling Rules

`#disable` turns a single rule **off** by its number; `#enable` turns it back on. A disabled rule is skipped every cycle — nothing else changes.

`/list` numbers the rules in its left column and marks a disabled one with `!`:

```
> /list
  1 R   Btn ? Led = 1
  2 R   Temp > 30 ? Fan = 1
> #disable 2
```

```
> /list
  1 R   Btn ? Led = 1
  2 R!  Temp > 30 ? Fan = 1     # off
> #enable 2                     # back on
```

You can name a list or a range, and sweep with a range:

```
#disable 3 5 7        # several at once
#disable 2-6          # an inclusive range
#enable  1-99      # re-enable everything (the top is clamped to the last rule)
```

What it is good for

- **Building past a bug.** A rule misbehaves — it fights another rule, trips on a bad sensor, floods an output. Instead of stopping the whole program to edit and reflash, switch that one rule off and keep running:

  ```
  > #disable 4              # silence the offending rule
  > Fan = Temp > 25         # add a corrected replacement rule
  ```

  The rest of the program is untouched. You have *built past* the bug live, and can take your time on a proper fix.

- **Patching firmware without reflashing.** Rule numbers run straight through the baked ROM rules and on into the ones you add at the prompt, so `#disable 2` can turn off a **firmware** rule just as easily as an interactive one. Silence a ROM rule, add a RAM rule in its place, and the board behaves the new way — no rebuild, no upload.

- **Bisecting behaviour.** Not sure which rule causes an effect? Disable a range, confirm it stops, then re-enable rules a few at a time to find the culprit.

- **Commissioning and testing.** Bring a machine up one rule at a time: disable everything, enable rules as each subsystem is verified.

Things worth knowing

- A disable **follows its rule**. Rule numbers shift when you insert or remove a rule, but a disable is remembered by identity across those edits — it stays on the rule you switched off, not on whatever number it happened to have.
- Disables **persist**. `/save` writes them to EEPROM and `/load` restores them, so a board comes back up with the same rules switched off. (If the program has changed so much that the numbers no longer line up, the saved set is dropped with a message rather than switching off the wrong rules.)
- `#disable 9` when there is no rule 9 is an **error** — naming a rule that does not exist is almost always a typo. A *range* that overshoots (`2-99`) is read as "to the end" and simply clamped.
- The first 128 rules each carry a switch; rules beyond that always run.

Direct Evaluation

In interactive mode you can:

```
> X + 1             # evaluate expression
```

```
> X = 5              # assign value directly
#variable Y = 10     # add new declaration
Y = X * 2        # add new rule
```

Output Latch

The latch controls whether outputs are written to hardware. Like an electronic latch:

- `/latch on` - Hold (latch) current output values. Outputs are calculated internally but not written to pins.
- `/latch off` - Release the latch. Outputs flow through to hardware normally.

Interactive mode starts with latch ON (outputs frozen). This is a safety feature - you can experiment without affecting hardware. Use `/latch off` when ready to activate outputs.

For programmatic control in running code, use the `latch()` function:

```
latch(1) ? Fault       // hold outputs on fault condition
latch(0) ? !Fault      // release when fault clears
```

Arduino Examples

Blink - Blinking LED

The classic Arduino example in CandySpeak:

```
#digital Led out 13
#timer Blink 500 = 1

Led = ~Led, Blink = 1 ? timeout(Blink)
```

LED toggles every 500 milliseconds. `Blink = 1` restarts the timer.

Button with LED

Light LED when button is pressed:

```
#digital Button in pullup 2
#digital Led out 13

Led = !Button
```

Button is active-low (pullup), so `!Button` is true when pressed.

Debouncer

Filter out contact bounce from a button:

```
#module Debouncer
#timer T 50
#variable Raw in integer
```

```
#variable Out:1 out integer = 0
#variable Prev:1 integer = 0

T = 1 ? Raw != Prev
Prev = Raw, Out = Raw ? timeout(T)
#end

#digital Button in pullup 2
#digital Led out 13
#Debouncer db Raw <- !Button

Led = db.Out
```

The module waits 50ms after a change before accepting the new value.

Toggle - Toggle with Button

Press to toggle LED:

```
#module Debouncer
#timer T 50
#variable Raw in integer
#variable Out:1 out integer = 0
#variable Prev:1 integer = 0

T = 1 ? Raw != Prev
Prev = Raw, Out = Raw ? timeout(T)
#end

#digital Button in pullup 2
#digital Led out 13
#variable LedState:1 = 0
#Debouncer db Raw <- !Button

LedState = ~LedState ? rising(db.Out)
Led = LedState
```

Dimmer - PWM Control

Adjust brightness with two buttons:

```
#digital BtnUp in pullup 2
#digital BtnDown in pullup 3
#analog Dimmer:8 out pwm 9
#variable Brightness:8 = 128
#timer Rep 100

Brightness = Brightness + 10 ? !BtnUp && Brightness < 245
Brightness = Brightness - 10 ? !BtnDown && Brightness > 10
```

```
Rep = 1 ? !BtnUp || !BtnDown
Dimmer = Brightness
```

Temperature Control

Simple thermostat with hysteresis:

```
#analog Sensor in A0
#digital Heater out 7
#variable Setpoint = 200
#variable Hysteresis = 10

Heater = 1 ? Sensor < Setpoint - Hysteresis
Heater = 0 ? Sensor > Setpoint + Hysteresis
```

The value 200 corresponds to approximately 20C with a typical NTC sensor.

Running Lights - Knight Rider

LEDs that move back and forth:

```
#digital Led0 out 2
#digital Led1 out 3
#digital Led2 out 4
#digital Led3 out 5
#digital Led4 out 6
#digital Led5 out 7
#digital Led6 out 8
#digital Led7 out 9

#timer Step 100 = 1
#variable Pos:4 = 0
#variable Dir:1 = 0

Pos = Pos + 1, Dir = 1 ? timeout(Step) && !Dir && Pos < 7
Pos = Pos - 1, Dir = 0 ? timeout(Step) && Dir && Pos > 0
Dir = 1 ? Pos == 7
Dir = 0 ? Pos == 0
Step = 1 ? timeout(Step)

Led0 = (Pos == 0)
Led1 = (Pos == 1)
Led2 = (Pos == 2)
Led3 = (Pos == 3)
Led4 = (Pos == 4)
Led5 = (Pos == 5)
Led6 = (Pos == 6)
Led7 = (Pos == 7)
```

Breathing LED

Smooth pulsing with PWM:

```
#analog Led:8 out pwm 9
#timer Step 20 = 1
#variable Brightness:8 = 0
#variable Dir:1 = 0

Brightness = Brightness + 5 ? timeout(Step) && !Dir
Brightness = Brightness - 5 ? timeout(Step) && Dir
Dir = 1 ? Brightness >= 250
Dir = 0 ? Brightness <= 5
Step = 1 ? timeout(Step)
Led = Brightness
```

Traffic Light

Sequential traffic light:

```
#digital Red out 2
#digital Yellow out 3
#digital Green out 4

#timer Phase 1000 = 1
#variable State:2 = 0

State = 1 ? timeout(Phase) && State == 0
State = 2 ? timeout(Phase) && State == 1
State = 3 ? timeout(Phase) && State == 2
State = 0 ? timeout(Phase) && State == 3
Phase = 1 ? timeout(Phase)

Red = (State == 0 || State == 1)
Yellow = (State == 1 || State == 3)
Green = (State == 2)
```

Sequence: Red -> Red+Yellow -> Green -> Yellow -> Red...

The same machine reads far more clearly with named `#states` — each phase is a state, and a transition is one assignment to `State`:

```
#digital Red    out 2
#digital Yellow out 3
#digital Green  out 4
#timer   Phase  1000 = 1
#states  red redyellow green yellow

#in INIT
    State = red
#end
```

```
#in red
    Red=1, Yellow=0, Green=0
    State = redyellow ? timeout(Phase)
#end
#in redyellow
    Red=1, Yellow=1, Green=0
    State = green ? timeout(Phase)
#end
#in green
    Red=0, Yellow=0, Green=1
    State = yellow ? timeout(Phase)
#end
#in yellow
    Red=0, Yellow=1, Green=0
    State = red ? timeout(Phase)
#end

Phase = 1 ? timeout(Phase)      // restart the timer each phase
```

Each state drives the three lamps and, on `timeout(Phase)`, hands over to the next state. The output pattern for a phase settles the cycle after the state is entered — invisible for a one-second light, and the structure scales to far more states without the tangle of `State == n` conditions.

Baking a Program into Firmware

A CandySpeak program can be *baked into the firmware* instead of being parsed at start-up. `-C` writes the parsed program as C source — `rom.c`, which defines the `rom_decl[]`, `rom_instr[]` and `rom_str[]` arrays:

```
./csp -C -n program.csp > rom.c
```

This `rom.c` is **compiled and linked** into the firmware (not `#included` as a header). At start-up `csp_load_rom()` points the runtime at it, and the program runs in place from flash.

What this buys you — and what it doesn't. The baked program runs on the *same* virtual machine over the *same* bytecode as a program typed at runtime; it is **not** faster (uncached flash reads can even make it a touch slower than a RAM-resident program). The real gains are elsewhere:

- **Saves RAM** — the program lives in flash, not in the scarce RAM.
- **No parsing at boot** — the program is ready immediately; the parser need not even be linked in.
- **It is firmware** — it survives without an EEPROM save.

Writing a Sketch

There is no auto-generated `.ino` wrapper. Today there are two practical paths, both starting from a fork/checkout of the CandySpeak sources:

1. **Hack the board glue.** Edit `csp_arduino.c` (the platform layer: `csp_board_*`, `setup()`/`loop()`). This is the easiest route when you are adding or wiring up hardware. See `CandySpeak/CandySpeak.ino` for a complete, working example (Circuit Playground Express) — it shows the real cycle: `csp_input()` → `csp_cycle()` → `csp_commit()` → `csp_output()`, wrapped in `csp_rt_init` / `csp_load_rom` / `csp_rt_start` / `csp_setup(&state)` during `setup()`.
2. **Freeze a program.** Generate `rom.c` with `-C` as above, drop it into the build, and flash. The sketch loads it via `csp_load_rom()`.

The firmware build needs the core sources — `csp.h`, `csp_config.h`, `csp_rt.c`, `csp_print.c`, `csp_strings.c`, `csp_parse.c`, `csp_eeprom.c`, `bitpack.h`, `csp_fixpoint.h` — plus your board glue and, for the frozen-program route, `rom.c`.

> This is the current, hands-on workflow; a smoother sketch story is still to be designed.

PDF Generation

This manual can be converted to PDF with pandoc:

```
# Install pandoc and LaTeX
sudo apt install pandoc texlive-xetex fonts-dejavu

# Generate PDF
pandoc doc/manual_en.md -o doc/manual_en.pdf \
  --pdf-engine=xelatex \
  --toc \
  --toc-depth=2 \
  -V colorlinks=true \
  -V linkcolor=blue
```

Appendix: Quick Reference

Declarations

```
#variable <name>[:<bits>] [type] [= value]          // bits 1..32, typeless = signed
#variable  <name>:<bits>  [big|little]  bind  <buffer>[<a>..<b>]     //  bit-
field view
#digital <name> [in|out|inout] [pullup|pulldown] [<port>:]<pin>
#analog <name>[:<resolution>] [in|out] [pwm] [<port>:]<pin>
#timer <name> <period_ms> [= 1]
#constant <name> = <value>
#buffer <name>:<bytes> [type]           // shared storage (size in BYTES)
#buffer <name>:<bytes> [in|out] can <id>  // CAN frame (size in BYTES)
#field <name>:<bits> [type] [big|little] <frame>[<a>..<b>] // field of a frame
#states <name> ...                      // INIT/NORMAL/FAILSAFE implicit
#module <name> ... #end
```

Buffers

```
#buffer Buf:3             // 3 BYTES of shared storage (size is always bytes)
Buf[0] = 52                 // byte access (index = byte)
X = Buf[0..1]               // byte range -> one value (little-endian)

#variable F:8     bind Buf[0..7]    // bind range = bits
#variable G:10 big bind Buf[8..17]  // big-endian bit-field
```

CAN

```
#buffer F201:8 in  can 0x201     // 8 BYTES (the DLC), received
#buffer Out:8  out can 0x200     // transmitted
#field Speed:16 unsigned F201[0..15]   // field: a bit view into the frame

Speed = v                      // writing a field sends the frame (event PDO)
Out.tx  = 1 ? timeout(Beat)      // cyclic PDO: send even if unchanged
Out.dlc = 3                      // send 3 bytes instead of 8

println(Speed) ? F201.rx        // .rx: true the cycle the frame is readable
Got = F201.dlc                   // bytes the sender actually sent
Id  = F201.id                    // frame id

./csp --can=vcan0 prog.csp       // Linux: SocketCAN
```

Module Instantiation

```
#<Module> <instance> [<init>]*

Init forms (can be mixed):
  field = value      // static init of the field's VALUE, runs once in INIT
  field.part = value // set a field attribute once
  field <- expr      // reactive connection (seeded once at start-up)

  D.pin = 2        // place a #digital/#analog  (NOT `D = 2`, that is a value)
  D.port = 1
  T.period = 500
```

Rules

```
<actions> ? <condition>
action1, action2 ? condition
variable = expression ? condition   // regular assignment
variable <- expression              // reactive (runs on change)
variable <- expression ? condition  // reactive with condition
Timer = 1 ? condition               // start timer
```

Timer Functions

```
timeout(T)    // true one cycle at timeout
elapsed(T)    // ms since start
progress(T)   // 0-100% of period
T = 1         // start/restart timer
```

Edge Detection

```
changed(x)    // value changed
rising(x)     // 0 -> 1
falling(x)    // 1 -> 0
```

States

```
#states A B C                // INIT, NORMAL and FAILSAFE always exist

#in A                        // rules active only while State == A
    ...
    State = B ? cond         // transition
#end

#in FAILSAFE                 // the safe island: entered once, never left
    ...                      // (only a reset releases it)
#end
```

Parts

```
<resource>.<part>            // read or write an attribute
D.pin = 17                   // .val .pin .port .dir .pwm .endian
T.period = 500               // .pullup .pulldown .period .fired
Ready = 1 ? T.fired
F.id  F.rx  F.tx  F.dlc      // CAN frame parts (also via any of its fields)
```

Interactive

```
/pause  /resume  /live       // freeze all / continue / freeze rules only
/latch on|off                // hold or release outputs
/list  /state  /memory       // inspect
/save  /load                 // EEPROM
#disable 2   #enable 2       // switch a rule off / on by number
#disable 2-6                 // a list or inclusive range
```

Packing and Unpacking

```
Buf <<= A B C             // pack fields into a buffer (ascending bits)
Buf <<= (X+4):3 X:2 6:3   // field := <expr>[:<bits>]
Buf >>= RA:3 RB:2 RC:3    // unpack: each var takes the next field
```